PyTorch Tip: Checkpoints

Are you running out of memory when training heavy PyTorch models? Do you not mind waiting for results?

Use checkpoints to trade time for memory.

The Problem: Memory Storage

Whenever you do an operation on a tensor, PyTorch stores a copy of it in memory.

```
import torch

# This is what we see...
x = torch.tensor(0.0, requires_grad=True)

x = x + 1   # x = 1
x = x * 2   # x = 2
x = x ** 2  # x = 4
x = x - 1   # x = 3
```

```
# ... but this is what PyTorch sees
x0 = torch.tensor(0.0, requires_grad=True)

x1 = x0 + 1   # x1 = 1 and x0 = 0
x2 = x1 * 2   # x2 = 2 and x1 = 1 and x0 = 0
x3 = x2 ** 2  #
```

The Problem: Heavy Functions

This is a problem when you're working with functions that have many operations.

```python
import torch

def heavy_function(x, a):
    for _ in range(100):
        x = x + a
    return x

class HeavyModel(torch.nn.Module):
    def __init__(self):
        super(HeavyModel, self).__init__()
        self.a = torch.nn.Parameter(torch.tensor(1.0))

    def forward(self, x):
        for _ in range(1000):
            x = heavy_function(x, self.a)
        return x

# This block will store 100,000 intermediate tensors
model = HeavyModel()
x = torch.tensor(0.0, requires_grad=True)
y = model(x)
y.backward()
```

The Solution: Use Checkpoints

Use checkpoints to trade time for memory!

Functions inside a checkpoint wrapper will not store intermediate tensors. Instead, PyTorch will recompute the tensors when you call the `backward()` method.

```
from torch.utils.checkpoint import checkpoint

# Wrap the function inside a checkpoint
x = checkpoint(heavy_function, x, self.a)
```

Implementing Checkpoints

Checkpoints are simple to use. Just wrap the function and its arguments inside a checkpoint.

```
import torch
from torch.utils.checkpoint import checkpoint

def heavy_function(x, a):
    for _ in range(100):
        x = x + a
    return x

class LightModel(torch.nn.Module):
    def __init__(self):
        super(LightModel, self).__init__()
        self.a = torch.nn.Parameter(torch.tensor(1.0))

    def forward(self, x):
        for _ in range(1000):
            x = checkpoint(heavy_function, x, self.a)
        return x

# This will store only about 100 intermediate tensors at a time
model = LightModel()
x = torch.tensor(0.0, requires_grad=True)
y = model(x)
y.backward()
# But it has to compute them twice: forward and backward pass
```

Wrap-Up

Now you can use checkpoints to trade time for memory when training models.

Follow me for more tips.
Shep Bryan IV